Gap-Aware Retrieval (GAR)

A Gap-Aware Reranking Algorithm for Iterative Retrieval-Augmented Generation

Abstract

Retrieval systems commonly rank documents according to their relevance to the original query. This can be insufficient in iterative information-seeking tasks, where the system already has some evidence and needs to retrieve only the information that is still missing. This project proposes **Gap-Aware Retrieval (GAR)**, a reranking algorithm that reorders a first-stage candidate set using three signals: relevance to the question and information gap, coverage of the missing information, and novelty with respect to previously collected evidence.

GAR is evaluated inside the existing MIGRES framework, which identifies missing information during iterative retrieval. MIGRES is the experimental framework; a first-stage retriever performs first-stage retrieval from the corpus, and GAR is the proposed reranking component applied on top of its candidates. Rather than assuming BM25 as the first-stage retriever, we select it empirically: BM25, DPR, Contriever, ColBERTv2, and SPLADE are each evaluated as standalone first-stage retrievers on a held-out validation set, and the retriever with the highest candidate-pool recall, denoted $R^\star$, is frozen as the fixed first-stage retriever for the remainder of the study. The surrounding MIGRES components are kept fixed as far as the original implementation permits. GAR is compared against $R^\star$ with no reranking and against ten established reranking algorithms — cross-encoder reranking (MonoBERT), MonoT5, pairwise reranking (DuoBERT), ColBERTv2 late-interaction reranking, classical learning-to-rank (LambdaMART), neural learning-to-rank (RankT5), Maximal Marginal Relevance (MMR), coverage-driven reranking (AIR), relevant-information-gain reranking (Dartboard/RIG), and LLM-based listwise reranking (RankGPT) — on the datasets used in the MIGRES evaluation: WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, and StrategyQA.

The study evaluates retrieval quality, downstream answer quality, ablations, efficiency, and statistical significance. GAR has no trainable parameters in the proposed implementation. Its scoring hyperparameters are selected on a held-out validation set using Bayesian optimization and then frozen before test evaluation. The formulation is motivated by prior work on missing-information-guided retrieval, iterative evidence coverage, relevant information gain, sentence embeddings, and efficient nearest-neighbor search [Wang et al., 2025, Yadav et al., 2020, Pickett et al., 2024, Reimers and Gurevych, 2019, Johnson et al., 2017].

1 Introduction

Retrieval-Augmented Generation (RAG) systems use external documents to provide a language model with information that may not be available in its parameters. A retrieval algorithm therefore has a simple but important job: given a question and a document collection, return a small set of useful documents.

A common retrieval objective is query–document relevance:

$$q + D \longrightarrow d_1, d_2, \ldots, d_k.$$

This works well when the question can be answered from one retrieval step. It becomes less direct when the system retrieves information iteratively. After one step, some parts of the information need may already be satisfied. The next retrieval step should focus on the remaining gap rather than repeatedly retrieving documents about the same topic.

MIGRES provides a useful setting for this problem. MIGRES is a Missing Information Guided Retrieve-Extraction-Solving framework. It identifies missing information after evidence is collected and uses that missing information to guide subsequent retrieval [Wang et al., 2025]. In this project, we do not modify the MIGRES reasoning, extraction, or solving framework. Instead, we replace its retrieval component with GAR.

The central research question is:

$$\boxed{\text{Does explicitly modeling the current information gap improve retrieval?}}$$

GAR addresses this by scoring a candidate document using:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q, G_t) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

where R measures relevance, C measures gap coverage, and N measures evidence novelty.

The contribution of this project is the proposed GAR reranking formulation, applied on top of $R^\star$'s first-stage retrieval, and its controlled empirical evaluation. The individual ideas used to construct GAR are grounded in previous work; the project does not claim that semantic relevance, information coverage, or novelty are individually new.

1.1 What is MIGRES and How Does It Work?

MIGRES stands for *Missing Information Guided Retrieve-Extraction-Solving*. It is an iterative framework designed for questions that require multiple pieces of information. Instead of retrieving documents only once, MIGRES repeatedly checks what information is already available, identifies what is still missing, retrieves information for that gap, and uses the retrieved information to continue solving the question [Wang et al., 2025].

The basic idea is:

$$\boxed{\text{Question} \rightarrow \text{Find Missing Information} \rightarrow \text{Retrieve} \rightarrow \text{Extract} \rightarrow \text{Update Information} \rightarrow \text{Repeat if Needed}}$$

In simple terms, MIGRES asks three questions during the process:

1. **What do we already know?** It keeps the evidence collected so far.
2. **What information is still missing?** It identifies the missing information needed to answer the question.
3. **What should we retrieve next?** It uses the missing information to guide the next retrieval step.

For example, suppose a question requires two facts, A and B. After the first retrieval, MIGRES may obtain A but determine that B is still missing. It then focuses the next retrieval step on B:

$$\text{Question} \rightarrow \text{retrieve } A \rightarrow \text{missing } B \rightarrow \text{retrieve } B \rightarrow \text{answer}.$$

MIGRES therefore maintains an iterative state consisting of the original question q, previously collected evidence E_t, and the current missing-information requirements G_t. These are provided to the retrieval component at each iteration:

$$\boxed{(q, E_t, G_t, D)}$$

where D is the searchable document corpus.

In our work, **MIGRES is the surrounding framework, not our contribution**. We use its iterative missing-information and evidence-generation process, while replacing its retrieval component with our proposed GAR algorithm. MIGRES determines *what information is missing*; GAR determines *which documents are most useful for obtaining that information.*

Thus, the division of responsibility is:

$$\boxed{\underbrace{\text{MIGRES}}_{\text{finds what is missing}} \rightarrow \underbrace{\text{GAR}}_{\text{reranks useful documents}} \rightarrow \underbrace{\text{MIGRES}}_{\text{extracts and updates evidence}}}$$

This allows us to test whether a gap-aware reranking algorithm improves the ranking stage while keeping the rest of the MIGRES framework, including $R^\star$ first-stage retrieval, unchanged.

1.2 What We Use from MIGRES

We use MIGRES as the surrounding experimental framework. MIGRES produces the iterative state:

$$(q, E_t, G_t, D),$$

where:

- q is the original question;
- E_t is evidence collected before retrieval iteration t;
- G_t is the current missing-information state;
- D is the searchable document corpus.

This state is used in two stages. First, $R^\star$ — the first-stage retriever selected empirically in Section 2.9 — performs first-stage retrieval from D using q and each requirement in G_t, producing a candidate set $\mathcal{C}_t$. Second, GAR reranks $\mathcal{C}_t$ using q, E_t, and G_t:

$$T_t = \text{TopK}_{d \in \mathcal{C}_t} S(d \mid q, E_t, G_t).$$

MIGRES then continues its normal filtering, extraction, gap-analysis, and solving process.

We therefore compare:

$$\boxed{\text{MIGRES}+R^\star \text{ (no rerank)}} \quad \text{vs.} \quad \boxed{\text{MIGRES}+R^\star+\text{GAR}}.$$

We additionally compare GAR with ten other established reranking methods, each applied on the same $R^\star$ candidate set $\mathcal{C}_t$ (Section 3). $R^\star$ is not assumed to be BM25: Section 2.9 describes how it is chosen from among the five first-stage retrievers introduced in Section 2 before any reranking comparison begins.

1.3 What We Do Not Change

The following MIGRES components should remain fixed wherever the original implementation permits:

- question processing;

- information-gap generation;
- query-generation logic;
- knowledge filtering;
- information extraction;
- answer-generation/solving model;
- stopping conditions;
- prompts and model settings;
- dataset and corpus.

The first-stage retriever $R^\star$ is selected once, empirically, and then held fixed like the other components above; the main experimental variable is therefore the reranking algorithm applied on top of it.

2 Information Retrieval Algorithms

2.1 What is an Information Retrieval Algorithm?

An *Information Retrieval (IR) algorithm* is a method used to search a collection of documents and rank the documents according to how useful they are for a given information need. In our setting, the information need is represented by a question or retrieval query q, and the IR algorithm produces an ordered list of documents:

$$\boxed{(q, D) \rightarrow d_1, d_2, \ldots, d_k}$$

where D is the document collection and $d_1, \ldots, d_k$ are the top-k retrieved documents.

Different IR algorithms use different signals to decide which documents should be ranked highly. For example, BM25 primarily uses lexical term-matching, DPR and Contriever use dense vector representations, ColBERTv2 uses token-level late interaction, and SPLADE uses sparse neural representations [Robertson and Zaragoza, 2009, Karpukhin et al., 2020, Izacard et al., 2022, Santhanam et al., 2022, Formal et al., 2021].

In a conventional retrieval system, the main objective is:

$$\boxed{\text{Find documents that are relevant to the query.}}$$

However, in an iterative system such as MIGRES, relevance to the original question may not be sufficient. Some information may already have been retrieved, while other information is still missing. This motivates GAR, which additionally considers the current information gap and the evidence already collected.

Thus, the difference between the first-stage retrieval algorithms and GAR is not that GAR is simply another embedding model. GAR is a reranking method whose scoring function explicitly incorporates the current information state, applied to the candidates $R^\star$ already retrieved:

$$\boxed{\text{Question} + \text{Information Gap} + \text{Existing Evidence} \rightarrow \text{Document Ranking.}}$$

2.2 Why Compare Retrieval Algorithms?

These five methods represent the established landscape of first-stage retrieval. Rather than assuming which one should feed the reranking comparison, Section 2.9 evaluates all five head-to-head and selects the fixed first-stage retriever, $R^\star$, empirically. GAR itself is not one of these five: it is a reranking algorithm applied on top of $R^\star$'s output, and its own comparison against established reranking algorithms is introduced in Section 3. The primary first-stage retrieval systems considered here are:

BM25, DPR, Contriever, ColBERTv2, SPLADE

These five methods are used through their published or official implementations where possible, as detailed in Section 2.8.

2.3 BM25

BM25 is a sparse probabilistic retrieval method based mainly on query-term frequency, inverse document frequency, and document-length normalization [Robertson and Zaragoza, 2009].

A common form is:

$$S_{\text{BM25}}(q,d) = \sum_{t \in q} IDF(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\frac{|d|}{avgdl}\right)}.$$

Here $f(t,d)$ is the frequency of term t in document d, $|d|$ is document length, $avgdl$ is average document length, and k_1, b are BM25 parameters.

BM25 is especially important because it is the retrieval baseline used in the MIGRES setting, which makes it one of the five candidates evaluated for selection as $R^\star$ in Section 2.9 rather than a default assumption.

2.4 DPR

Dense Passage Retrieval (DPR) uses separate dense encoders for questions and passages and retrieves documents according to their vector similarity [Karpukhin et al., 2020].

A simplified score is:

$$S_{\text{DPR}}(q,d) = \mathbf{q}^T \mathbf{d},$$

where $\mathbf{q}$ and $\mathbf{d}$ are dense query and document representations.

DPR provides a supervised dense-retrieval baseline.

2.5 Contriever

Contriever is an unsupervised dense retrieval method based on contrastive learning [Izacard et al., 2022]. Its basic retrieval operation can be written as:

$$S_{\text{Contriever}}(q,d) = sim(\mathbf{q}, \mathbf{d}).$$

Contriever provides a dense-retrieval baseline with a different training approach from DPR.

2.6 ColBERTv2

ColBERTv2 uses late interaction. Instead of representing a complete document with only one vector, it keeps token-level representations and computes efficient token-level interactions [Santhanam et al., 2022].

A simplified score is:

$$S_{\text{ColBERT}}(q, d) = \sum_{i \in q} \max_{j \in d} sim(\mathbf{q}_i, \mathbf{d}_j).$$

This gives a useful comparison with both single-vector dense retrieval and sparse retrieval.

2.7 SPLADE

SPLADE is a neural sparse retrieval method that produces sparse vocabulary representations for queries and documents [Formal et al., 2021].

Let:

$$\mathbf{s}_d = f_\theta(d)$$

be a sparse document representation. A simplified retrieval score is:

$$S_{\text{SPLADE}}(q, d) = \mathbf{s}_q^T \mathbf{s}_d.$$

SPLADE provides a neural sparse baseline.

2.8 Baseline Comparison

System	Retrieval type	Role
MIGRES+BM25	Sparse lexical	Candidate for $R^\star$ selection
MIGRES+DPR	Dense bi-encoder	Candidate for $R^\star$ selection
MIGRES+Contriever	Unsupervised dense	Candidate for $R^\star$ selection
MIGRES+ColBERTv2	Late interaction	Candidate for $R^\star$ selection
MIGRES+SPLADE	Neural sparse	Candidate for $R^\star$ selection

All five should use established implementations and published checkpoints where possible. Their exact checkpoint, index, and retrieval parameters must be recorded. GAR does not appear in this table: it is not a first-stage retriever, but a reranking algorithm applied on top of $R^\star$'s output, whichever of the five above that turns out to be. GAR's own comparison, against ten established reranking algorithms, is given in Section 3.

2.9 Selecting the First-Stage Retriever $R^\star$

The reranking comparison in Section 3 needs a single, fixed first-stage retriever to feed every reranker the same candidate set $\mathcal{C}_t$. Rather than assuming this should be BM25 because it is the retriever MIGRES originally used, we select it empirically, using the same logic as GAR's own hyperparameter selection (Section 6.13.4): choose it on validation data, before any reranking results are inspected, and then freeze it for the rest of the study.

2.9.1 Selection Protocol

For each candidate retriever $\rho \in \{\text{BM25}, \text{DPR}, \text{Contriever}, \text{ColBERTv2}, \text{SPLADE}\}$, we run ρ alone — no reranking — on D_{val} and measure candidate-pool recall at the same depth K_0 that will be used throughout the main experiment (note: ρ here denotes a candidate retriever, not the relevance term $R(d, q, G_t)$ from Section 5.4):

$$\boxed{\text{Recall@}K_0(\rho) = \frac{1}{|D_{val}|} \sum_{q \in D_{val}} \frac{|\text{relevant evidence} \cap \rho_{K_0}(q, D)|}{|\text{relevant evidence}|}}$$

This is deliberately a retrieval-only measurement: it requires no reranking, no answer generation, and no additional LLM calls, so all five candidates can be evaluated cheaply and in parallel. The selected retriever is the one with the highest validation Recall@K_0:

$$\boxed{R^\star = \arg\max_{\rho} \ \text{Recall@}K_0(\rho)}$$

$R^\star$ is chosen once, on D_{val}, and then frozen: it is not re-selected per dataset, per reranker, or on D_{test}, for the same reason GAR's own hyperparameters are frozen before test evaluation (Section 6.13.5).

2.9.2 Why Recall@K_0, Not Downstream Accuracy

Every reranker evaluated in Section 3, including GAR, can only reorder $\mathcal{C}_t$ — it cannot introduce a document the first-stage retriever failed to retrieve. Recall@K_0 measures exactly this ceiling: how much relevant evidence is even available to be reranked. Running the complete MIGRES pipeline five times (once per candidate retriever) to compare downstream EM/F1 instead would multiply the LLM cost of the whole study just to make this one selection, without changing which quantity actually determines the reranking comparison's upper bound. Downstream answer quality for the selected $R^\star$ is still reported in full as part of the main experiment (Section 8); it is only the *selection step* that uses the cheaper retrieval-only metric.

2.9.3 An Efficiency Caveat

If $R^\star$ turns out to be BM25, the efficiency profile described in Sections 5–6 holds unchanged: first-stage retrieval needs no embeddings and no corpus-wide index. If $R^\star$ instead turns out to be a dense or late-interaction method (DPR, Contriever, ColBERTv2, or SPLADE), first-stage retrieval itself now requires a corpus-wide embedding index, and the indexing time, index memory, and preprocessing cost reported in Section 8.8 must include that cost, not just GAR's on-demand embedding of $\mathcal{C}_t$. Recall@K_0 decides *which* retriever is used, but it should not be read in isolation from this cost; both are reported together in the final selection record.

3 Information Reranking Algorithms

3.1 What is Reranking?

A *reranking algorithm* is the second stage of a two-stage retrieval pipeline. A first-stage retriever (here, $R^\star$, selected empirically in Section 2.9) narrows a large corpus D down to a small candidate set $\mathcal{C}_t$ using a cheap, high-recall method. A reranker then re-scores and reorders only that candidate set using a signal that would be too expensive or too fine-grained to run over the full corpus D directly:

$$\boxed{(q, \mathcal{C}_t) \to d_1, d_2, \ldots, d_k, \qquad \mathcal{C}_t \subseteq D, \quad |\mathcal{C}_t| \ll |D|}$$

Reranking never introduces documents that the first-stage retriever missed; it can only reorder what was already retrieved. This is the key structural difference from a full retrieval algorithm, which must search D itself.

Example. Suppose the first-stage retriever returns 100 passages for the question "What year was the treaty signed?" because they share lexical terms with the query ("treaty", "signed") — this is the typical behavior of a lexical retriever such as BM25, one of the five candidates for $R^\star$ in Section 2.9. Many of these 100 passages discuss the treaty's causes, its signatories, or its consequences without ever stating the year. A reranker re-examines only these 100 candidates and reorders them so that passages actually containing a year near the word "treaty" are pushed to the top, even if their raw first-stage score was not the highest.

3.2 Why Compare Reranking Algorithms?

GAR reranks the candidate set $\mathcal{C}_t$ returned by $R^\star$, so the correct comparison is against established reranking algorithms operating on the same candidate set, rather than against full first-stage retrievers. The primary systems are:

MonoBERT, MonoT5, DuoBERT, ColBERTv2 (reranker), LambdaMART RankT5, MMR, AIR, Dartboard / RIG, RankGPT, GAR

Each is applied to the identical $R^\star$ candidate set $\mathcal{C}_t$, so only the reranking method differs across systems.

3.3 Cross-Encoder Reranking (MonoBERT)

MonoBERT jointly encodes the question and a candidate passage as a single input to a pretrained BERT model, and uses the resulting classification score as the relevance score [Nogueira and Cho, 2019].

A simplified score is:

$$S_{\text{MonoBERT}}(q, d) = \sigma(f_\theta([q; d])),$$

where f_θ is a fine-tuned BERT classifier and σ is the sigmoid function. Because the question and document are encoded jointly, MonoBERT captures fine-grained term interactions that bi-encoder retrieval methods cannot, at the cost of requiring a full forward pass per candidate — practical only because $|\mathcal{C}_t|$ is small.

3.4 MonoT5

MonoT5 reframes reranking as a sequence-to-sequence task: the model is given the question and candidate passage as input text and is trained to generate the token `true` or `false` [Nogueira et al., 2020].

The reranking score is the model's probability assigned to the `true` token:

$$S_{\text{MonoT5}}(q, d) = P(\texttt{true} \mid q, d).$$

MonoT5 provides a pointwise reranking baseline from a different model family than MonoBERT.

3.5 DuoBERT (Pairwise Reranking)

DuoBERT reframes reranking as pairwise comparison: given the question and two candidate documents, the model predicts which of the two is more relevant, rather than scoring each candidate independently [Nogueira et al., 2019].

$$S_{\text{DuoBERT}}(q, d_i, d_j) = P(d_i \succ d_j \mid q), \qquad d_i, d_j \in \mathcal{C}_t,$$

and a candidate's final score aggregates its pairwise win probability against the other candidates in $\mathcal{C}_t$. Pairwise comparison can capture relative distinctions between similar candidates that a pointwise score misses, at the cost of $O(|\mathcal{C}_t|^2)$ comparisons instead of $O(|\mathcal{C}_t|)$.

3.6 ColBERTv2 as a Reranker

ColBERTv2 keeps token-level representations and computes late-interaction similarity between the question and a candidate document [Santhanam et al., 2022]. In Section 2 it was described as a candidate for $R^\star$; here the identical scoring function is instead applied only to the $R^\star$ candidate set $\mathcal{C}_t$ (note that if ColBERTv2 itself is selected as $R^\star$, this reranking pass is redundant with first-stage retrieval and should be skipped for that one system):

$$S_{\text{ColBERT}}(q, d) = \sum_{i \in q} \max_{j \in d} sim(\mathbf{q}_i, \mathbf{d}_j), \qquad d \in \mathcal{C}_t.$$

This provides a direct comparison between a multi-vector late-interaction reranker and GAR's single-score fusion of relevance, coverage, and novelty.

3.7 LambdaMART

LambdaMART is a classical gradient-boosted-tree learning-to-rank algorithm, trained on hand-engineered features (term overlap, document length, the candidate's prior $R^\star$ score, and similar signals) rather than learned text representations [Burges, 2010].

$$S_{\text{LambdaMART}}(q, d) = \sum_{\text{trees}} w_{\text{tree}}(\mathbf{x}(q, d)), \qquad d \in \mathcal{C}_t,$$

where $\mathbf{x}(q, d)$ is a hand-engineered feature vector rather than a learned embedding. LambdaMART is included as a non-neural learning-to-rank baseline, since it remains a strong, cheap production reranker in practice and does not depend on the Sentence Transformer model GAR and the other neural baselines share.

3.8 Maximal Marginal Relevance (MMR)

MMR reranks candidates by balancing relevance to the query against redundancy with documents already selected into the output list [Carbonell and Goldstein, 1998]. It is the closest existing baseline to GAR's own relevance-versus-novelty design.

The greedy selection rule is:

$$S_{\text{MMR}}(d) = \lambda \, sim(d, q) - (1 - \lambda) \max_{d_j \in S} sim(d, d_j), \qquad d \in \mathcal{C}_t \setminus S,$$

where S is the set of documents already selected and sim is a similarity function. MMR has no explicit gap-coverage term: it balances only relevance and redundancy, whereas GAR separately scores relevance, gap coverage, and evidence novelty.

3.9 AIR (Coverage-Driven Iterative Evidence Retrieval)

AIR reformulates its query after each retrieval step by focusing on keywords not yet covered by the evidence already retrieved, so that later steps target complementary evidence rather than repeating earlier ones [Yadav et al., 2020]. AIR directly motivated GAR's coverage term C (Section 4). Adapted here as a reranker rather than its original iterative-query design, AIR scores each $R^\star$ candidate by how many of the gap's uncovered keywords it contains:

$$S_{\text{AIR}}(d, G_t) = \frac{|\tau(d) \cap \tau_{\text{uncovered}}(G_t)|}{|\tau_{\text{uncovered}}(G_t)|}, \qquad d \in \mathcal{C}_t,$$

where $\tau_{\text{uncovered}}(G_t)$ is the set of gap tokens not already present in E_t. Unlike GAR, AIR has no separate relevance or novelty term — coverage is its only signal.

3.10 Dartboard / RIG (Relevant Information Gain)

Dartboard selects passages that maximize the total relevant information gained from the retrieved set, explicitly penalizing redundancy with information already collected [Pickett et al., 2024]. This directly motivated GAR's novelty term N (Section 4). As a reranker over $\mathcal{C}_t$, candidates are selected greedily by information gain relative to E_t:

$$S_{\text{Dartboard}}(d \mid E_t) = \text{Rel}(d, q) - \max_{e \in E_t} sim(d, e), \qquad d \in \mathcal{C}_t,$$

which is structurally close to GAR's $\alpha R + \gamma N$ terms with $\beta = 0$ — Dartboard combines relevance and novelty but has no explicit gap-coverage term of its own.

3.11 RankT5

RankT5 fine-tunes a T5 model with ranking losses (pairwise or listwise) so that the model directly outputs a scalar ranking score for each candidate, rather than a classification probability [Zhuang et al., 2023].

$$S_{\text{RankT5}}(q, d) = f_\theta(q, d), \qquad d \in \mathcal{C}_t,$$

where f_θ is trained end-to-end with a ranking objective rather than a pointwise classification loss. RankT5 represents the learning-to-rank family of rerankers.

3.12 RankGPT (LLM Listwise Reranking)

RankGPT prompts a large language model with the question and the full list of candidate passages at once, and asks the model to output a reordering of that list directly, rather than scoring each candidate independently [Sun et al., 2023].

$$(d_1, \ldots, d_{|\mathcal{C}_t|}) \xrightarrow{\text{LLM listwise prompt}} \pi(d_1, \ldots, d_{|\mathcal{C}_t|}),$$

where π is the model-produced permutation. RankGPT is included because MIGRES is itself LLM-driven, making an LLM-based reranking baseline a natural comparison point.

3.13 Reranking Baseline Comparison

System	Reranking type	Role
MIGRES+$R^\star$	No reranking	Control
MIGRES+$R^\star$+MonoBERT	Cross-encoder pointwise	Reranking baseline
MIGRES+$R^\star$+MonoT5	Seq2seq pointwise	Reranking baseline
MIGRES+$R^\star$+DuoBERT	Pairwise	Reranking baseline
MIGRES+$R^\star$+ColBERTv2	Late interaction	Reranking baseline†
MIGRES+$R^\star$+LambdaMART	Classical learning-to-rank	Reranking baseline
MIGRES+$R^\star$+RankT5	Neural learning-to-rank	Reranking baseline
MIGRES+$R^\star$+MMR	Relevance vs. redundancy	Closest conceptual baseline
MIGRES+$R^\star$+AIR	Gap-coverage-driven	Direct precursor to C
MIGRES+$R^\star$+Dartboard/RIG	Relevance vs. novelty	Direct precursor to N
MIGRES+$R^\star$+RankGPT	LLM listwise	Reranking baseline
MIGRES+$R^\star$+GAR	Gap-aware reranking	Proposed method

†If ColBERTv2 itself is selected as $R^\star$, this row is dropped, since reranking $R^\star$'s own candidates with $R^\star$'s own scoring function is redundant.

All rerankers are applied to the identical $R^\star$ candidate set $\mathcal{C}_t$ for each question, so that only the reranking method varies across systems. AIR and Dartboard/RIG are included not merely as baselines but as the two prior works that directly motivated GAR's coverage and novelty terms; the comparison therefore also tests whether GAR's fused three-signal score outperforms each of its own inspirations used alone.

4 Development and Introduction of GAR

4.1 What GAR Is

GAR is a reranking algorithm designed for iterative retrieval-augmented generation. It reranks the candidate set $\mathcal{C}_t$ returned by $R^\star$'s first-stage retrieval, rather than searching the corpus D directly. It changes the ranking question from:

"Is this document relevant to the original question?"

to:

"Is this document useful given what is missing and what is already known?"

GAR uses three signals:

1. **Gap relevance**: relevance to the original question and the current information gap.
2. **Gap coverage**: how much of the missing information is covered by the candidate.
3. **Evidence novelty**: how much new information the candidate adds beyond existing evidence.

4.2 Why GAR Was Developed

A query-only retriever can repeatedly return documents that are strongly related to the question but do not resolve the current missing information. MIGRES already exposes the missing-information state, making it possible for the retrieval algorithm to use that state directly [Wang et al., 2025].

The motivation for the GAR components comes from three sources.

First, MIGRES provides the missing-information concept: later retrieval can be targeted toward information that is still missing [Wang et al., 2025].

Second, Yadav et al. use an iterative evidence-retrieval process that focuses later retrieval on information not covered by existing justifications. This motivates the coverage component in GAR [Yadav et al., 2020].

Third, Relevant Information Gain argues that retrieval should avoid wasting limited context on redundant information while retaining relevance [Pickett et al., 2024]. This motivates the novelty component.

GAR combines these ideas into one retrieval score. The formulation is therefore a proposed unified model; the project does not claim that each component is independently novel.

4.3 Applications

GAR is intended for settings where information needs change during retrieval, including:

- multi-hop question answering;
- iterative RAG;
- research-oriented evidence retrieval;
- evidence gathering over large document collections;
- systems that explicitly track missing information.

GAR does not require a new language model. The surrounding system can provide the current gap and evidence state.

5 Mathematical Formulation

5.1 Notation

At retrieval iteration t:

$$q = \text{original question},$$

$$E_t = \text{evidence collected before iteration } t,$$

$$G_t = \{g_1, \ldots, g_m\}$$

is the set of current information requirements, and

$$D = \text{searchable document corpus.}$$

$R^\star$ performs first-stage retrieval over D and returns a candidate set:

$$\mathcal{C}_t \subseteq D, \qquad |\mathcal{C}_t| = K_0.$$

A candidate document is a member of this candidate set, not of the full corpus directly:

$$d \in \mathcal{C}_t.$$

An individual evidence item is:

$$e \in E_t.$$

The GAR input is therefore:

$$\boxed{(q, E_t, G_t, \mathcal{C}_t)}.$$

GAR never receives D directly; D is used only by $R^\star$ to construct $\mathcal{C}_t$ (Section 6).

5.2 Text Embeddings

GAR needs a fixed representation function:

$$\phi : \mathcal{T} \to \mathbb{R}^p.$$

We use:

`sentence-transformers/multi-qa-mpnet-base-cos-v1`

as the fixed embedding model. The model maps sentences and paragraphs to 768-dimensional dense vectors and was designed for semantic search [Reimers and Gurevych, 2019, Sentence Transformers, 2020].

The same model is used for questions, gaps, candidate documents, and evidence. Candidate document embeddings are computed on demand for $\mathcal{C}_t$; GAR itself never maintains a corpus-wide embedding index for this purpose. Whether *first-stage retrieval* needs a corpus-wide index depends on which retriever is selected as $R^\star$ (Section 2.9.3): none is needed if $R^\star$ is BM25, but one is if $R^\star$ is DPR, Contriever, ColBERTv2, or SPLADE.

5.3 Cosine Similarity

For two text items x and y:

$$\cos(x, y) = \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}.$$

For a normalized score in $[0, 1]$, GAR uses:

$$\boxed{s(x, y) = \frac{1}{2}\left(1 + \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}\right)} \tag{1}$$

The embedding model creates vectors; cosine similarity compares them.

5.4 Gap Relevance

All information requirements in G_t are treated as equally important. First calculate the semantic relevance to the gap by averaging similarity across all requirements:

$$\boxed{s_G(d, G_t) = \frac{1}{m} \sum_{i=1}^{m} s(d, g_i)} \tag{2}$$

Then combine original-question relevance and gap relevance:

$$\boxed{R(d, q, G_t) = \lambda s(d, q) + (1 - \lambda) s_G(d, G_t)} \tag{3}$$

where:

$$0 \leq \lambda \leq 1.$$

Interpretation:

- λ close to 1: emphasize the original question;
- λ close to 0: emphasize the information gap.

5.5 Gap Coverage

Let $\tau(x)$ be the set of normalized content tokens obtained after the fixed preprocessing pipeline.

For a non-empty gap requirement:

$$\boxed{c(d, g_i) = \frac{|\tau(d) \cap \tau(g_i)|}{|\tau(g_i)|}} \tag{4}$$

This is normalized token recall. The coverage score is the average recall across all gap requirements:

$$\boxed{C(d, G_t) = \frac{1}{m} \sum_{i=1}^{m} c(d, g_i)} \tag{5}$$

Thus $C(d, G_t) \in [0, 1]$.

If $\tau(g_i)$ is empty, that invalid requirement is removed before retrieval.

5.6 Evidence Novelty

For non-empty evidence:

$$\boxed{N(d, E_t) = 1 - \max_{e \in E_t} s(d, e)} \tag{6}$$

A candidate very similar to existing evidence receives low novelty. A candidate dissimilar to all existing evidence receives high novelty.

At the first iteration:

$$E_0 = \emptyset.$$

We define:

$$\boxed{N(d, \emptyset) = 1.} \tag{7}$$

5.7 Final GAR Score

The final score is:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q, G_t) + \beta C(d, G_t) + \gamma N(d, E_t)} \tag{8}$$

subject to:

$$\boxed{\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1.} \tag{9}$$

Because the three components are normalized, the final score is also in $[0, 1]$.

5.8 Hyperparameters

GAR has four scoring hyperparameters:

$$\boxed{\alpha, \beta, \gamma, \lambda}$$

and one retrieval hyperparameter:

$$\boxed{K_0}.$$

Their roles are:

Parameter	**Role**	**Constraint**
α	Weight of relevance R	≥ 0
β	Weight of coverage C	≥ 0
γ	Weight of novelty N	≥ 0
λ	Question-vs-gap relevance balance	$[0, 1]$
K_0	$R^\star$ candidate-pool size (first-stage retrieval depth)	$K_0 \geq k$

The final retrieval depth k is an **experimental setting**, not a GAR-specific scoring parameter. It should be kept the same across systems for fair comparison.

The fixed embedding model and cosine similarity are also configuration choices, not learned GAR hyperparameters.

6 How GAR Works Using MIGRES

6.1 Overall Architecture

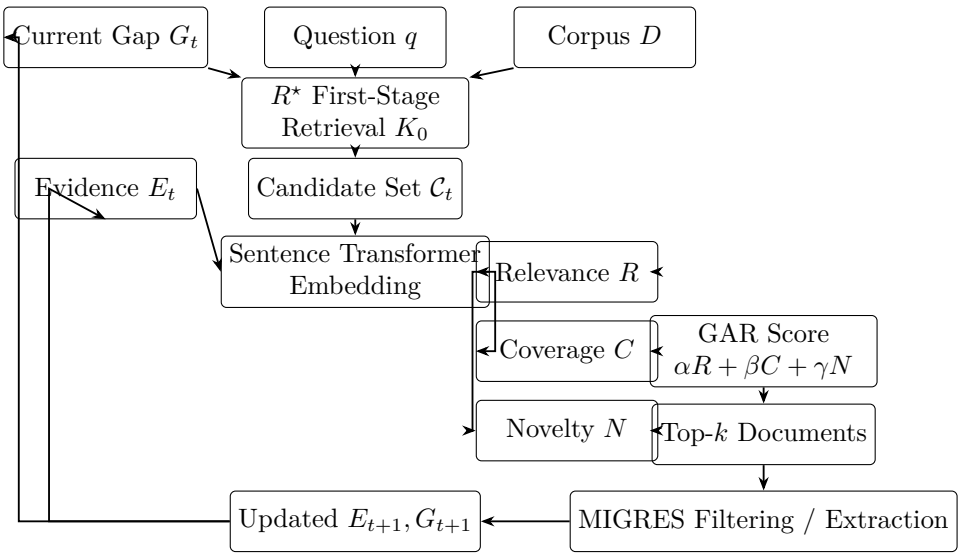


Figure 1: Overall architecture of GAR within the MIGRES framework. MIGRES provides the current question, evidence, and information gap. $R^\star$ — the first-stage retriever selected empirically in Section 2.9 — performs first-stage retrieval over the corpus D to produce the candidate set $\mathcal{C}_t$; only these candidates are embedded and reranked by GAR using relevance, gap coverage, and evidence novelty. The reranked documents are then returned to MIGRES for filtering, extraction, and state updating.

6.2 Step 1: Dataset Question

A benchmark supplies the original question:

$$q.$$

The benchmark's gold answer and supporting evidence are not given to GAR during retrieval.

6.3 Step 2: MIGRES Creates the Retrieval State

MIGRES processes the question and maintains:

$$E_t = \text{evidence already collected}$$

and:

$$G_t = \text{information still missing.}$$

The gap can contain multiple requirements:

$$G_t = \{g_1, \ldots, g_m\}.$$

GAR does not independently invent G_t in the main experiment. It receives the gap produced by MIGRES.

6.4 Step 3: First-Stage Retrieval ($R^\star$)

$R^\star$ — the first-stage retriever selected empirically in Section 2.9, from among BM25, DPR, Contriever, ColBERTv2, and SPLADE — performs first-stage retrieval on D. For each query-like input x, $R^\star$ returns the K_0 highest-scoring documents:

$$R^\star_{K_0}(x, D).$$

GAR's candidate set is built using both the original question and every gap requirement, so that first-stage retrieval is itself gap-aware:

$$\boxed{\mathcal{C}_t = R^\star_{K_0}(q, D) \cup \bigcup_{i=1}^{m} R^\star_{K_0}(g_i, D)} \tag{10}$$

Duplicates are removed. Thus K_0 is the candidate-pool size, while k is the final number returned. If $R^\star$ is BM25, this step is a cheap lexical lookup requiring no embeddings or dense index, even over a corpus D with millions of documents. If $R^\star$ is instead DPR, Contriever, ColBERTv2, or SPLADE, this step requires querying a corpus-wide embedding or sparse-vocabulary index built ahead of time; that index's build time and memory footprint are then part of the pipeline's fixed cost, not GAR's, and are reported alongside GAR's own (much smaller) on-demand embedding cost in Section 8.8.

6.5 Step 4: Convert Candidate Text to Embeddings

GAR uses the fixed Sentence Transformer model:

$$\phi(x).$$

The question, each gap requirement, each evidence item, and *only the documents already in* $\mathcal{C}_t$ are represented as vectors. Because $|\mathcal{C}_t| = K_0 \ll |D|$, GAR itself never needs to embed or index the full corpus for its own scoring — embeddings are computed on demand for the small candidate set returned by $R^\star$, regardless of whether $R^\star$ needed its own corpus-wide index to produce that candidate set.

6.6 Step 5: Calculate Relevance

For every candidate:

$$R(d, q, G_t)$$

measures both:

question-document relevance

and:

gap-document relevance.

6.7 Step 6: Calculate Coverage

GAR then asks:

"Does this document cover the information that is missing?"

This produces:

$$C(d, G_t).$$

6.8 Step 7: Calculate Novelty

GAR then asks:

"Is this document redundant with evidence already collected?"

This produces:

$$N(d, E_t).$$

6.9 Step 8: Calculate the GAR Score

For every candidate:

$$S(d \mid q, E_t, G_t) = \alpha R + \beta C + \gamma N.$$

The candidate with the highest score is preferred because it is simultaneously relevant, gap-covering, and non-redundant.

6.10 Step 9: Return Top-k

GAR returns:

$$\boxed{T_t = \mathrm{TopK}_{d \in \mathcal{C}_t} S(d \mid q, E_t, G_t)} \tag{11}$$

where k is fixed for the experiment.

6.11 Step 10: MIGRES Continues

MIGRES receives the retrieved documents and performs its normal filtering and extraction.
It updates:

$$E_{t+1}$$

and, if more information is needed:

$$G_{t+1}.$$

GAR is then called again.
The complete loop is:

$$\boxed{(q, E_t, G_t, D) \xrightarrow{R^\star} \mathcal{C}_t \xrightarrow{\text{GAR}} T_t \xrightarrow{\text{MIGRES}} (E_{t+1}, G_{t+1}) \xrightarrow{R^\star} \mathcal{C}_{t+1} \xrightarrow{\text{GAR}} T_{t+1}}$$

This is why GAR is evaluated inside MIGRES rather than as an isolated one-shot reranker.

6.12 Retrieval vs. Reranking in This Design

The two stages must not be confused:

$$\boxed{R^\star \text{ candidate generation} \rightarrow \text{Embedding} \rightarrow \text{GAR scoring} \rightarrow \text{Top-}k}$$

$R^\star$ finds a manageable candidate set from the full corpus D.
Embedding converts only that candidate set (plus q, G_t, E_t) into vectors.
GAR makes the final ranking decision over the candidate set.
This is precisely the reranking design proposed in this project: $R^\star$ performs first-stage retrieval, and GAR reranks its output using relevance, gap coverage, and evidence novelty. A version of GAR that performs its own first-stage candidate generation directly from D (for example, via an ANN index over GAR's own embeddings, without relying on $R^\star$) would instead be a complete end-to-end retrieval algorithm; that variant is not evaluated here and is left as a direction for future work.

6.13 Development, Validation, and Test Protocol

6.13.1 Is GAR Trained?

No.
The proposed GAR algorithm has no trainable parameters. We do not train GAR on D_{train}.
The embedding model is a fixed pretrained Sentence Transformer. It is not fine-tuned in the primary experiment.
Therefore the experiment has no GAR training stage.

6.13.2 Training Data

If a baseline uses an official pretrained checkpoint, we use that checkpoint rather than retraining it on the evaluation data.
The benchmark training split may be used for auxiliary development or baseline-specific training only if required by the official implementation. Any such use must be documented and must not use test annotations.
For GAR itself:

$$D_{\text{train}} \not\to \text{GAR parameter learning.}$$

6.13.3 Validation Set D_{val}

We define:

$$\boxed{D_{\text{val}} = \text{the held-out development/validation split.}}$$

Where an official development split exists, we use it. The exact files and split names are recorded for each dataset.
GAR hyperparameters are selected using the *complete MIGRES+$R^\star$+GAR pipeline* on D_{val}, after $R^\star$ itself has already been selected and frozen (Section 2.9).
This is important because GAR cannot be meaningfully validated in isolation: E_t and G_t are produced by the MIGRES iterative process, and $\mathcal{C}_t$ is produced by $R^\star$.
The validation loop is:

$$D_{\text{val}} \to \text{MIGRES} \to (q, E_t, G_t) \to R^\star \to \mathcal{C}_t \to \text{GAR} \to \text{MIGRES} \to \text{validation metric.}$$

6.13.4 Bayesian Optimization

We use Bayesian optimization to select:

$$\theta_{\text{GAR}} = \{\alpha, \beta, \gamma, \lambda, K_0\}.$$

Bayesian optimization uses previous validation evaluations to decide which configuration to test next rather than exhaustively evaluating every possible combination [Snoek et al., 2012].
The search space is:

$$\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1,$$

$$0 \leq \lambda \leq 1,$$

and a predefined discrete candidate set for K_0, for example:

$$K_0 \in \{50, 100, 200, 500\}.$$

The exact search space and random seed are recorded before the experiment.

The optimization objective must be selected before test evaluation. The primary objective should be a retrieval metric aligned with the main research question, such as Recall@k on D_{val}.
The procedure is:

$$\boxed{D_{\text{val}} \rightarrow \text{MIGRES}{+}R^{\star}{+}\text{GAR} \rightarrow \text{validation score} \rightarrow \text{Bayesian optimization} \rightarrow \theta^{*}_{\text{GAR}}}$$

The selected configuration is then frozen.

6.13.5 Test Set D_{test}

After validation:

$$\boxed{\theta^{*}_{\text{GAR}} \text{ is frozen.}}$$

We then evaluate:

$$D_{\text{test}} \rightarrow \text{MIGRES}{+}R^{\star}{+}\text{GAR}(\theta^{*}_{\text{GAR}}) \rightarrow \text{final metrics.}$$

No test result is used to change the hyperparameters.
The same k, dataset, corpus, MIGRES configuration, and stopping conditions are used for the comparison systems wherever possible.

6.13.6 Complete Experimental Flow

1. Prepare the official dataset splits and searchable corpus.
2. Run BM25, DPR, Contriever, ColBERTv2, and SPLADE standalone on D_{val}; measure Recall@K_0 for each; select $R^{\star} = \arg\max_{\rho} \text{Recall@}K_0(\rho)$ and freeze it (Section 2.9).
3. Reproduce MIGRES+$R^{\star}$ as the fixed first-stage retriever and as the no-reranking control.
4. Verify the MIGRES retrieval interface and confirm how E_t and G_t are exposed.
5. Implement on-demand embedding of $R^{\star}$'s candidate set $\mathcal{C}_t$ using the fixed Sentence Transformer.
6. Implement and unit-test the GAR equations.
7. Run Bayesian optimization on D_{val} through the complete MIGRES+$R^{\star}$+GAR pipeline.
8. Freeze $\alpha, \beta, \gamma, \lambda, K_0$.
9. Run the held-out D_{test} with the frozen configuration.
10. Run MonoBERT, MonoT5, DuoBERT, ColBERTv2, LambdaMART, RankT5, MMR, AIR, Dartboard/RIG, and RankGPT as rerankers on the identical $R^{\star}$ candidate set, using their established implementations.
11. Evaluate retrieval quality, downstream answer quality, efficiency, and cost.
12. Run GAR ablations using the same frozen evaluation protocol.

7 Datasets

7.1 Dataset Selection

The project uses the six datasets reported in the MIGRES evaluation:

WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, StrategyQA

These datasets provide questions and task-specific answer/evidence annotations suitable for evaluating retrieval and downstream question answering.

7.2 WikiHop

Paper: Welbl et al. [2018].
WikiHop is a multi-document, multi-hop reading-comprehension benchmark designed so that the answer requires combining information across documents. The dataset is useful for GAR because it tests whether retrieval can assemble complementary evidence rather than repeatedly retrieving one topic.
Typical fields include a question, candidate documents, an answer, and supporting information.
Paper: https://aclanthology.org/Q18-1021/

7.3 HotpotQA

Paper: Yang et al. [2018].
HotpotQA contains about 113K Wikipedia-based question-answer pairs and provides sentence-level supporting facts. It includes multi-hop and comparison questions [Yang et al., 2018].
For first-stage development, we use the FullWiki setting because it requires retrieval from a larger Wikipedia corpus. The official repository provides train, development, and FullWiki resources and explicitly allows researchers to replace its retrieval component with their own IR system.
Paper: https://aclanthology.org/D18-1259/
Official resources: https://hotpotqa.github.io/
The repository documentation describes the FullWiki development and test files and the use of custom IR systems.

7.4 MuSiQue

Paper: Trivedi et al. [2022].
MuSiQue was designed to reduce shortcut solutions in multi-hop QA. MuSiQue-Ans contains about 25K questions requiring 2–4 hops [Trivedi et al., 2022].
Typical information includes a question, answer, reasoning structure, and supporting evidence.
Paper: https://aclanthology.org/2022.tacl-1.31/
Official repository: https://github.com/StonyBrookNLP/musique

7.5 Natural Questions

Paper: Kwiatkowski et al. [2019].
Natural Questions consists of real anonymized queries issued to Google Search and Wikipedia-based annotations. The public release contains 307,373 training examples, 7,830 development examples, and 7,842 sequestered test examples [Kwiatkowski et al., 2019].
Typical fields include a question, Wikipedia page information, long-answer annotations, short-answer annotations, and document-level information.
Paper: https://aclanthology.org/Q19-1026/
Official research page: https://ai.google/research/NaturalQuestions

7.6 TriviaQA

Paper: Joshi et al. [2017].
TriviaQA contains about 95K question-answer pairs and more than 650K question-answer-evidence triples. Questions were written by trivia enthusiasts and paired with independently gathered evidence documents [Joshi et al., 2017].
Typical fields include a question, answer, aliases, and evidence documents.
Paper: https://aclanthology.org/P17-1147/
Official resources: https://nlp.cs.washington.edu/triviaqa/

7.7 StrategyQA

Paper: Geva et al. [2021].
StrategyQA contains 2,780 questions with implicit reasoning strategies. Each example includes a question, a reasoning decomposition, and Wikipedia evidence paragraphs [Geva et al., 2021].
This makes it useful for testing whether gap-aware reranking helps when the required reasoning steps are not directly stated in the question.
Paper: https://aclanthology.org/2021.tacl-1.21/
Official resources: https://allenai.org/data/strategyqa

7.8 Dataset Summary

Dataset	Scale	Useful properties for GAR
WikiHop	∼51K	Multi-document, multi-hop reasoning.
HotpotQA	∼113K	Multi-hop QA with sentence-level supporting facts; FullWiki retrieval.
MuSiQue	∼25K	2–4 hop compositional reasoning.
Natural Questions	307,373 train; 7,830 dev; 7,842 test	Real search questions with Wikipedia annotations.
TriviaQA	∼95K QA pairs; >650K evidence triples	Heterogeneous evidence and lexical variation.
StrategyQA	2,780	Implicit reasoning with evidence paragraphs.

Exact counts used in experiments must be taken from the downloaded release and recorded in the experiment configuration. Dataset counts can differ by release or preprocessing.

7.9 What a Dataset Example Looks Like

The common conceptual structure is:

$$\boxed{\text{Question} + \text{Corpus/Evidence} + \text{Answer} + \text{Ground-Truth Evidence/Annotations}}$$

For example, a HotpotQA-style example contains a natural-language question, an answer, supporting facts, and Wikipedia context. During reranking, GAR receives the question, MIGRES's evidence and gap state, and the candidate set already retrieved by $R^\star$ from the searchable corpus. The gold answer and gold supporting evidence are reserved for evaluation.
The ground-truth evidence must never be used to construct G_t, E_t, or the $R^\star$ candidate pool during test-time retrieval.

7.10 Dataset Split Policy

For each dataset:

$$D_{\text{train}}, \quad D_{\text{val}}, \quad D_{\text{test}}$$

are treated separately where official splits are available.
For GAR:

$$D_{\text{val}}$$

is used for hyperparameter optimization, while:

$$D_{\text{test}}$$

is used only for the final evaluation.
If an official public test set has hidden answers, its evaluation follows the dataset's official evaluation procedure.

8 Experiments

8.1 Research Questions

RQ0. Among established first-stage retrievers, which one provides the best candidate pool (highest Recall@K_0) for the downstream reranking comparison, and how large is the recall gap between the selected $R^\star$ and the runner-up?
RQ1. Does reranking $R^\star$'s candidates with GAR improve retrieval quality over the MIGRES $R^\star$ baseline with no reranking?

$$\boxed{\text{MIGRES}+R^\star+\text{GAR} \;>\; \text{MIGRES}+R^\star?}$$

RQ2. Does GAR outperform established reranking methods applied to the same $R^\star$ candidates?

GAR vs. {MonoBERT, MonoT5, DuoBERT, ColBERTv2, LambdaMART, RankT5, MMR, AIR, Dartboard/

RQ3. Does reranking $R^\star$'s output with GAR improve downstream MIGRES answer quality, compared with no reranking?
RQ4. Which GAR component contributes most: relevance, coverage, or novelty?
RQ5. Does GAR generalize across the six datasets?
RQ6. What is the computational cost of GAR compared with the baselines?
RQ7. Does GAR improve retrieval without adding unnecessary LLM calls?

8.2 Hypotheses

RQ0 is answered descriptively, via the selection protocol in Section 2.9, rather than through significance testing: it determines $R^\star$ itself and is settled before any of the hypotheses below are evaluated.
Primary hypothesis:

H_0 : GAR reranking does not improve performance over $R^\star$ with no reranking.

H_1 : GAR reranking improves performance over $R^\star$ with no reranking.

Secondary hypothesis:

H_0 : GAR $\leq$ best reranking baseline.

H_1 : GAR $>$ best reranking baseline.

For ablations:

H_0 : removing a GAR component does not change performance.

H_1 : removing a GAR component reduces performance.

We report effect sizes and confidence intervals together with statistical significance. The exact paired statistical test should be fixed before inspecting the final test results; a paired bootstrap or paired permutation procedure is suitable for per-question retrieval metrics.

8.3 Main Comparison

The main experiment compares:

MIGRES+$R^\star$ (no rerank)
MIGRES+$R^\star$+MonoBERT
MIGRES+$R^\star$+MonoT5
MIGRES+$R^\star$+DuoBERT
MIGRES+$R^\star$+ColBERTv2
MIGRES+$R^\star$+LambdaMART
MIGRES+$R^\star$+RankT5
MIGRES+$R^\star$+MMR
MIGRES+$R^\star$+AIR
MIGRES+$R^\star$+Dartboard/RIG
MIGRES+$R^\star$+RankGPT
MIGRES+$R^\star$+GAR

The same questions, corpus, k, K_0, MIGRES settings, answer-generation model, and stopping conditions should be used wherever possible, so that only the reranking method varies across systems. $R^\star$ itself is fixed for every row in this table, having already been selected once in Section 2.9 (RQ0) and never re-selected per reranker or per dataset.

8.4 Ablation Studies

The full GAR model is:

$$S_{\text{GAR}} = \alpha R + \beta C + \gamma N.$$

We remove one component at a time.
Without relevance:

$$\boxed{S_{-R}(d) = \beta C(d, G_t) + \gamma N(d, E_t)}$$

Without coverage:

$$\boxed{S_{-C}(d) = \alpha R(d, q, G_t) + \gamma N(d, E_t)}$$

Without novelty:

$$\boxed{S_{-N}(d) = \alpha R(d, q, G_t) + \beta C(d, G_t)}$$

We also recommend two additional diagnostic ablations:

$$GAR_{\text{Q-only}}$$

which uses only question relevance, and:

$$GAR_{\text{Q+G}}$$

which uses question and gap relevance but not novelty.
These help determine whether the main benefit comes from simply adding the gap representation or from the complete relevance–coverage–novelty combination.

8.5 Candidate-Pool Sensitivity

Because K_0 controls how many candidates reach the full GAR scorer, we test:

$$K_0 \in \{50, 100, 200, 500\}$$

on the validation set during optimization.
The test experiment uses only the selected value.
We report both retrieval quality and latency as K_0 changes.

8.6 Retrieval Metrics

8.6.1 Recall@k

Recall@k measures how much of the ground-truth relevant evidence appears in the first k retrieved documents:

$$\boxed{Recall@k = \frac{|\text{relevant evidence} \cap \text{top-}k|}{|\text{relevant evidence}|}}$$

This is a primary metric because GAR is designed to retrieve missing evidence.

8.6.2 Precision@k

$$\boxed{Precision@k = \frac{|\text{relevant documents in top-}k|}{k}}$$

Precision measures how much of the retrieved set is relevant.

8.6.3 MRR

Mean Reciprocal Rank measures how early the first relevant result appears:

$$\boxed{MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}}$$

where $rank_i$ is the rank of the first relevant result for question i.

8.6.4 nDCG@k

For graded relevance:

$$DCG@k = \sum_{i=1}^{k}\frac{2^{rel_i}-1}{\log_2(i+1)}$$

and:

$$\boxed{nDCG@k = \frac{DCG@k}{IDCG@k}}$$

where $IDCG@k$ is the ideal DCG at cutoff k [Järvelin and Kekäläinen, 2002].

8.6.5 Evidence Coverage

Because GAR explicitly models an information gap, we additionally report an evidence-coverage measure where the dataset annotations permit it:

$$\boxed{GapEvidenceCoverage@k = \frac{\text{ground-truth evidence requirements retrieved in top-}k}{\text{total ground-truth evidence requirements}}}$$

The exact mapping from benchmark annotations to evidence requirements must be defined separately for each dataset.

8.7 Downstream QA Metrics

The final MIGRES answer is evaluated using the metrics supported by each benchmark.
These may include:

- Exact Match (EM);
- token-level F1;
- answer accuracy for classification-style datasets;
- evidence/supporting-fact quality where annotations permit.

For a benchmark with a normalized answer set A_i:

$$EM_i = \mathbb{1}[\hat{a}_i \in A_i].$$

Average exact match is:

$$EM = \frac{1}{N} \sum_{i=1}^{N} EM_i.$$

For token-level F1, precision and recall are computed over normalized answer tokens and then combined as:

$$F1 = \frac{2PR}{P + R}.$$

The benchmark's official evaluation script should be used whenever available.

8.8 Efficiency Metrics

We report:

- retrieval latency per question;
- $R^\star$ indexing time (zero if $R^\star$ is BM25; nonzero if $R^\star$ is DPR, Contriever, ColBERTv2, or SPLADE, per the caveat in Section 2.9.3);
- index memory;
- number of retrieval iterations;
- number of documents scored by GAR;
- number of LLM calls in the complete MIGRES system;
- input and output tokens;
- API cost where applicable.

The GAR scorer itself should not require one LLM call per candidate. Embeddings, cosine similarity, lexical coverage, and novelty can be calculated locally.

8.9 API Cost

If an external model is used by MIGRES, the cost for question q is:

$$\boxed{Cost(q) = T_{\text{in}}(q)p_{\text{in}} + T_{\text{out}}(q)p_{\text{out}}}$$

where T_{in} and T_{out} are input and output token counts and $p_{\text{in}}, p_{\text{out}}$ are the corresponding prices. Average cost is:

$$\boxed{\overline{Cost} = \frac{1}{N}\sum_{i=1}^{N} Cost(q_i)}$$

The comparison must distinguish GAR's local retrieval computation from the LLM cost of the surrounding MIGRES framework.

8.10 Fairness Controls

The following should remain fixed across systems:

- dataset split;
- question set;
- document corpus;
- final retrieval depth k;
- MIGRES components outside retrieval;
- answer-generation model;
- stopping conditions;
- evaluation scripts;
- random seeds where applicable.

Only the retrieval method should change.

8.11 Reproducibility Configuration

The final experiment record should contain:

Parameter	Value
Dataset	Exact release and split
Corpus	Exact corpus version
$R^\star$ (first-stage retriever)	Selected on D_{val} by Recall@K_0 (Section 2.9); fixed for all systems
$R^\star$ selection Recall@K_0	Value for $R^\star$ and each runner-up (RQ0)
Reranker	None / MonoBERT / MonoT5 / DuoBERT / ColBERTv2 / LambdaMART / RankT5 / MMR / AIR / Dartboard-RIG / RankGPT / GAR
Baseline checkpoint	Exact model/checkpoint
GAR embedding model	`multi-qa-mpnet-base-cos-v1`
Final retrieval depth k	Fixed value for all systems
$R^\star$ candidate pool K_0	Validation-selected GAR value
α	Validation-selected value
β	Validation-selected value
γ	Validation-selected value
λ	Validation-selected value
Bayesian optimization seed	Recorded value
$R^\star$ implementation	Exact library/index
Hardware	CPU/GPU configuration
Software	Python and package versions
MIGRES configuration	Exact prompts/model/settings

9 Conclusion

This project proposes Gap-Aware Retrieval (GAR) as a reranking algorithm for iterative information-seeking systems, applied on top of $R^\star$'s first-stage retrieval. $R^\star$ is itself selected empirically from five established retrievers rather than assumed to be BM25 (Section 2.9). GAR is evaluated inside MIGRES rather than replacing the MIGRES framework itself.
The core idea is:

$$\boxed{\text{Relevant} + \text{Gap-covering} + \text{Non-redundant}}$$

implemented as:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q, G_t) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

The complete pipeline is:

$$\boxed{R^\star \text{ First-Stage Retrieval} \rightarrow \text{Sentence Transformer} \rightarrow \text{Cosine Similarity} \rightarrow \text{GAR Scoring} \rightarrow \text{Top-}k \rightarrow \text{MIGRES}}$$

$R^\star$ is selected once, empirically, from among BM25, DPR, Contriever, ColBERTv2, and SPLADE, using validation-set candidate recall (Section 2.9); it is then frozen and used identically for GAR and for every reranking baseline. GAR has no training stage in the primary experiment. Its scoring hyperparameters are selected on D_{val} through the complete MIGRES+$R^\star$+GAR pipeline using Bayesian optimization and then frozen for D_{test}.
The central empirical question is whether reranking with the information gap and previously collected evidence produces a more useful ranking than standard reranking algorithms applied to the same $R^\star$ candidates.

References

Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai. LLMs know what they need: Leveraging a missing information guided framework to empower retrieval-augmented generation.

In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 2379–2400. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.coling-main.163/.

Vikas Yadav, Steven Bethard, and Mihai Surdeanu. Unsupervised alignment-based iterative evidence retrieval for multi-hop question answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4514–4525. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.414. URL https://aclanthology.org/2020.acl-main.414/.

Marc Pickett, Jeremy Hartman, Ayan Kumar Bhowmick, Raquib-ul Alam, and Aditya Vempaty. Better rag using relevant information gain. *arXiv preprint arXiv:2407.12101*, 2024. URL https://arxiv.org/abs/2407.12101.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *arXiv preprint arXiv:1702.08734*, 2017. URL https://arxiv.org/abs/1702.08734.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019. URL https://doi.org/10.1561/1500000019.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL https://aclanthology.org/2020.emnlp-main.550/.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2665–2679. Association for Computational Linguistics, 2022. URL https://aclanthology.org/2022.emnlp-main.738/.

Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.272. URL https://aclanthology.org/2022.naacl-main.272/.

Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. *arXiv preprint arXiv:2107.05720*, 2021. URL https://arxiv.org/abs/2107.05720.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019. URL https://arxiv.org/abs/1901.04085.

Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. Document ranking with a pre-trained sequence-to-sequence model. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 708–718. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.63. URL https://aclanthology.org/2020.findings-emnlp.63/.

Rodrigo Nogueira, Wei Yang, Kyunghyun Cho, and Jimmy Lin. Multi-stage document ranking with BERT. *arXiv preprint arXiv:1910.14424*, 2019. URL https://arxiv.org/abs/1910.14424.

Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025. URL https://doi.org/10.1145/290941.291025.

Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. RankT5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2308–2313, 2023. doi: 10.1145/3539618.3592047. URL https://arxiv.org/abs/2210.10634.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14918–14937. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.923. URL https://aclanthology.org/2023.emnlp-main.923/.

Sentence Transformers. multi-qa-mpnet-base-cos-v1. Hugging Face model card, 2020. URL https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-cos-v1. 768-dimensional sentence-transformers model designed for semantic search.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25, 2012. URL https://arxiv.org/abs/1206.2944.

Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel. Constructing datasets for multi-hop reading comprehension across documents. *Transactions of the Association for Computational Linguistics*, 6:287–302, 2018. doi: 10.1162/tacl_a_00021. URL https://aclanthology.org/Q18-1021/.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1259. URL https://aclanthology.org/D18-1259/.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022. doi: 10.1162/tacl_a_00475. URL https://aclanthology.org/2022.tacl-1.31/.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611. Association for Computational Linguistics, 2017. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147/.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021. doi: 10.1162/tacl_a_00370. URL https://aclanthology.org/2021.tacl-1.21/.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418. URL https://doi.org/10.1145/582415.582418.